

Bài thuyết trình: Vibe Engineering

Nhóm 6

Thành viên nhóm:

Nguyễn Hải Đăng - 3122411031

Đình Trung Hội - 3122411058

Huỳnh Phúc Hưng - 3122411073

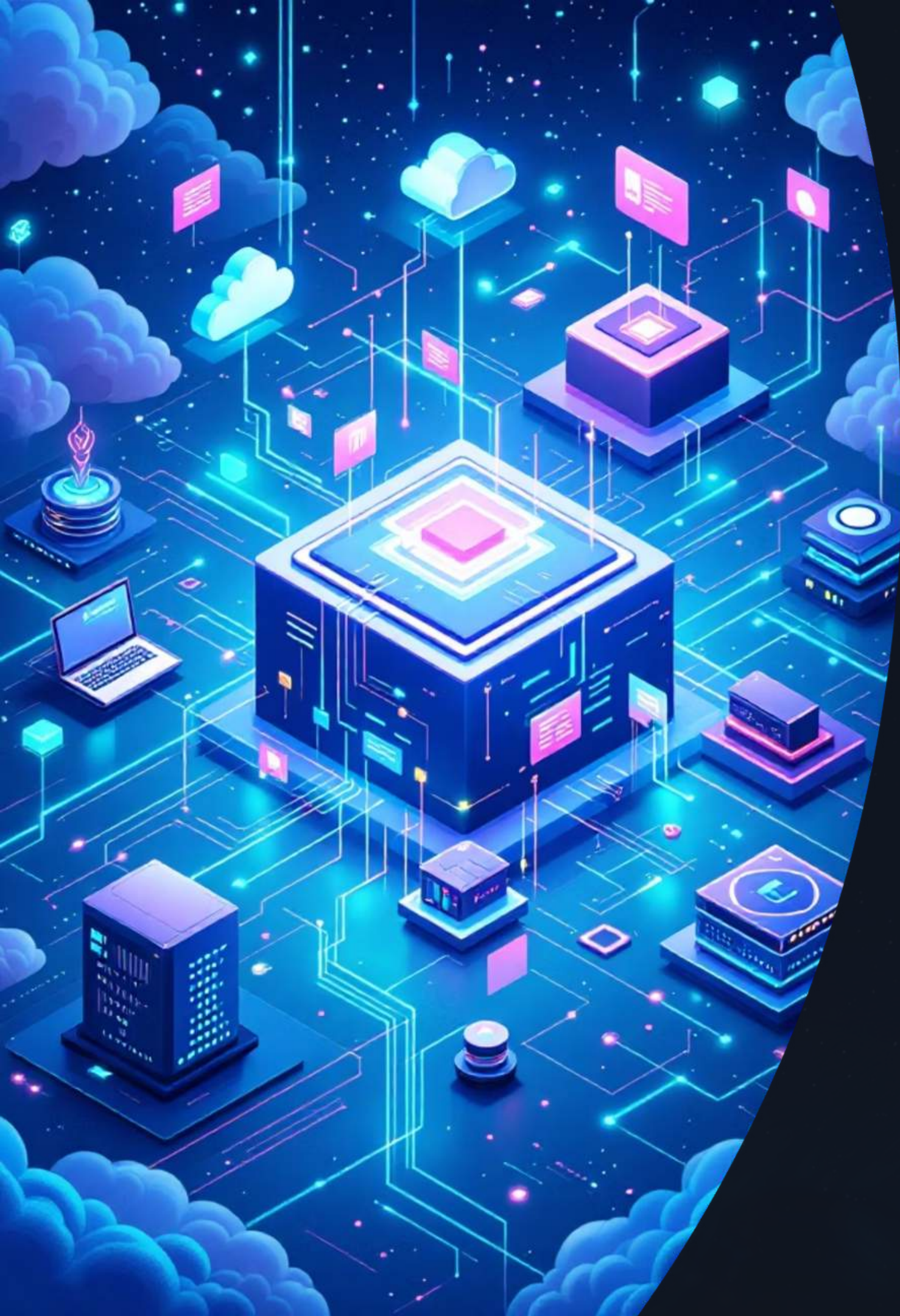
Bùi Văn Tiến - 3122411206



Phát triển Backend Microservices & Ứng dụng GenAI

Tổng quan & Kiến trúc hệ thống phân tán hiện đại





Bối cảnh & Lý do chọn đề tài

Chuyển đổi từ Monolithic sang Microservices

Kiến trúc monolithic truyền thống gặp nhiều hạn chế về khả năng mở rộng, bảo trì và triển khai độc lập. Microservices giải quyết bài toán này bằng cách chia nhỏ hệ thống thành các dịch vụ tự trị, có thể phát triển và vận hành riêng biệt.

Sự bùng nổ của GenAI trong kỹ nghệ phần mềm

Trí tuệ nhân tạo tạo sinh (Generative AI) đang cách mạng hóa quy trình phát triển phần mềm — từ hỗ trợ viết code, tạo tài liệu, đến tự động hóa kiểm thử và tối ưu hiệu năng hệ thống.

Mục tiêu dự án



Mục tiêu cốt lõi

Xây dựng hệ thống backend hiện đại, có khả năng mở rộng và tích hợp AI vào toàn bộ vòng đời phát triển phần mềm.

1 Backend NestJS phân tán

Xây dựng hệ thống backend bằng **NestJS** theo kiến trúc microservices phân tán, đảm bảo tính module và khả năng mở rộng.

2 Ứng dụng AI vào phát triển

Tích hợp **GenAI** vào vòng đời phát triển — hỗ trợ code generation, review tự động và tối ưu hiệu năng.

3 Triển khai containerized

Sử dụng **Docker** để đóng gói và triển khai từng service độc lập, đảm bảo tính nhất quán môi trường.

Công nghệ sử dụng

Stack công nghệ hiện đại được lựa chọn để xây dựng hệ thống microservices mạnh mẽ và linh hoạt.



NestJS

Framework backend TypeScript với kiến trúc module, hỗ trợ Dependency Injection và tích hợp dễ dàng với các thư viện khác.



Prisma ORM

Database toolkit thế hệ mới với type-safe queries, auto-completion và migration tự động cho nhiều loại database.



Docker

Containerization platform giúp đóng gói ứng dụng cùng dependencies, đảm bảo chạy nhất quán trên mọi môi trường.



MongoDB / PostgreSQL

Database linh hoạt — PostgreSQL cho dữ liệu có cấu trúc, MongoDB cho dữ liệu phi cấu trúc và scale ngang.

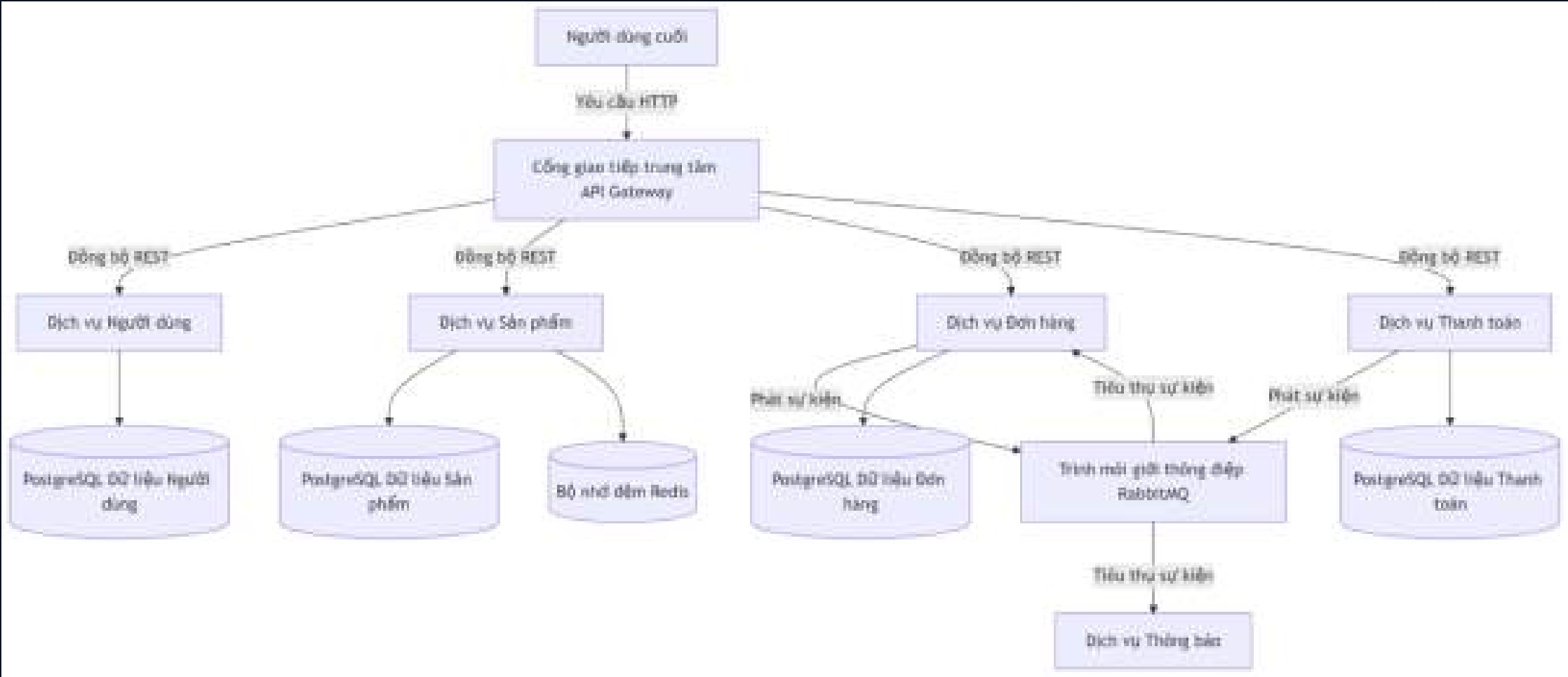


GenAI

Trí tuệ nhân tạo tạo sinh hỗ trợ code generation, phân tích lỗi, tạo tài liệu và tối ưu hiệu năng hệ thống.

Kiến trúc Microservices tổng thể

Hệ thống được thiết kế theo mô hình phân tán với API Gateway làm điểm truy cập duy nhất, định tuyến request đến các service nghiệp vụ độc lập.



Sơ đồ luồng thể hiện kiến trúc phân tán: Client gửi request đến API Gateway, Gateway định tuyến đến các service nghiệp vụ tương ứng, mỗi service quản lý database riêng biệt và giao tiếp thông qua Message Queue.

Độc lập

Mỗi service hoạt động và deploy độc lập

Cô lập

Database riêng biệt cho từng service

Giao tiếp

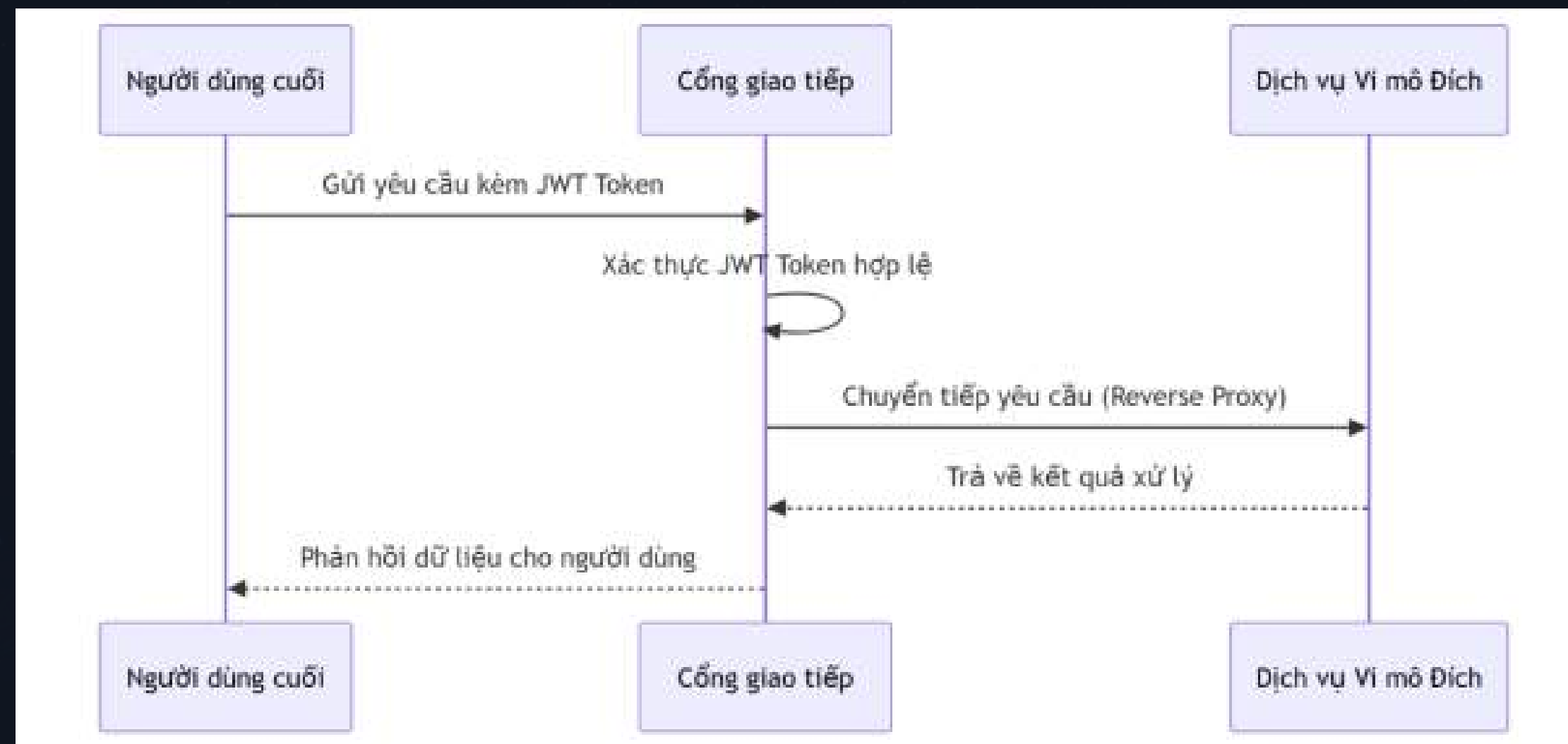
REST API & Message Queue

Container

Triển khai qua Docker

API Gateway — "Người điều phối"

API Gateway đóng vai trò trung gian duy nhất tiếp nhận mọi request từ client, đảm bảo bảo mật, định tuyến và tối ưu hiệu năng hệ thống.



Các Services nghiệp vụ chính

SERVICE

Order Service

Quản lý đơn hàng

- Tạo, cập nhật và hủy đơn hàng
- Quản lý trạng thái đơn hàng (pending, confirmed, shipped)
- Tích hợp với Payment Service
- Lưu trữ dữ liệu vào PostgreSQL
- Gửi event qua Message Queue

Notification Service

Gửi Email / SMS

- Gửi email xác nhận đơn hàng
- Gửi SMS thông báo trạng thái
- Quản lý template thông báo
- Lưu lịch sử gửi vào MongoDB
- Lắng nghe event từ Message Queue

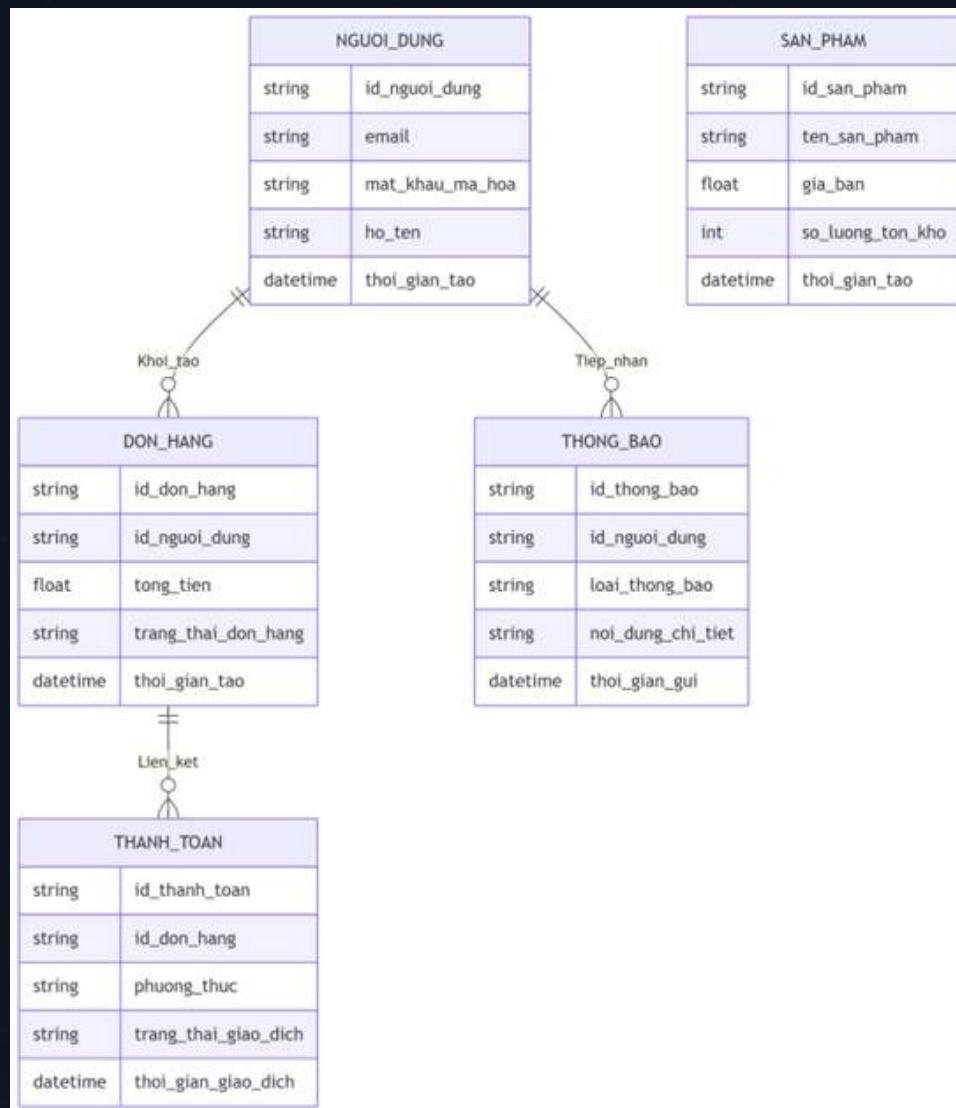


Notification Service hoạt động theo mô hình **event-driven** — không gọi trực tiếp mà lắng nghe event từ các service khác.

Thiết kế Cơ sở dữ liệu

Prisma ORM — Quản lý dữ liệu linh hoạt

Mỗi microservice quản lý database riêng biệt, đảm bảo tính cô lập và độc lập trong phát triển.



Schema mẫu — Prisma

```
// schema.prisma
model Order {
  id      String    @id @default(uuid())
  userId  String
  status  String    @default("pending")
  items   OrderItem[]
  createdAt DateTime @default(now())
}

model OrderItem {
  id          String    @id @default(uuid())
  orderId     String
  order       Order     @relation(fields: [orderId],
references: [id])
  productId   String
  quantity    Int
  price       Decimal
}
```

Prisma tạo tự động TypeScript types từ schema, giúp code an toàn và dễ bảo trì.

→ Type-safe Schema

Prisma Schema định nghĩa cấu trúc dữ liệu với full type support từ TypeScript.

→ Auto Migration

Tự động tạo migration file khi thay đổi schema, đồng bộ database an toàn.

→ Database per Service

Order Service dùng PostgreSQL, Notification Service dùng MongoDB — mỗi service độc lập.

Tổng kết & Bước tiếp theo

Phần 1 đã thiết lập nền tảng kiến trúc vững chắc. Phần 2 sẽ đi sâu vào triển khai thực tế và tích hợp GenAI.

Đã hoàn thành (Phần 1)

- Tổng quan kiến trúc Microservices
- Thiết kế API Gateway & Services
- Stack công nghệ: NestJS, Prisma, Docker
- Thiết kế Database phân tán

Sắp tới (Phần 2)

- Triển khai NestJS Microservices thực tế
- Tích hợp GenAI vào quy trình CI/CD
- Optimization & Performance Testing
- Deploy lên Cloud (AWS/GCP)

5

Công nghệ core

NestJS, Prisma, Docker, MongoDB, PostgreSQL

2

Services chính

Order Service & Notification Service

1

API Gateway

Điểm truy cập duy nhất & bảo mật

"Kiến trúc tốt không chỉ là viết code đúng — mà là xây dựng hệ thống có thể phát triển cùng tương lai."

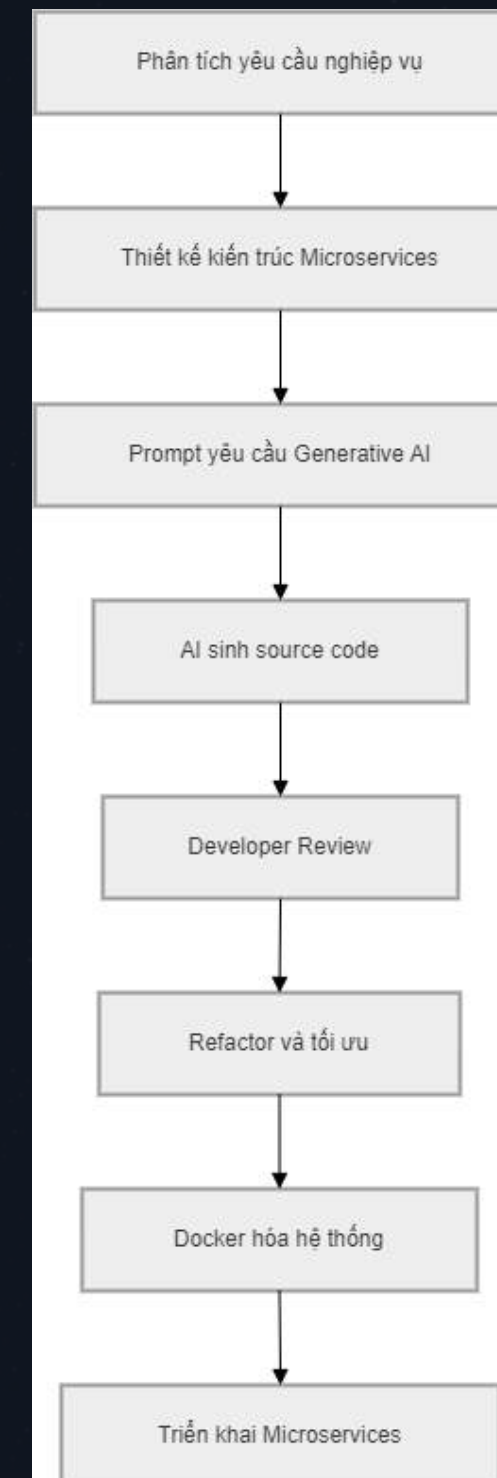
Phương pháp Vibe Coding, Triển khai Hệ thống và Đánh giá Kết quả

Hành trình khám phá cách lập trình viên hiện đại kết hợp tư duy kiến trúc với sức mạnh của AI để xây dựng hệ thống Microservices hiệu quả.



Khái niệm "Vibe Coding"

Vibe Coding là phương pháp lập trình hiện đại trong đó lập trình viên tập trung vào **tư duy thiết kế kiến trúc** và giải quyết vấn đề ở cấp độ cao, trong khi AI đảm nhận việc sinh mã nguồn cơ bản (Boilerplate code).



Developer

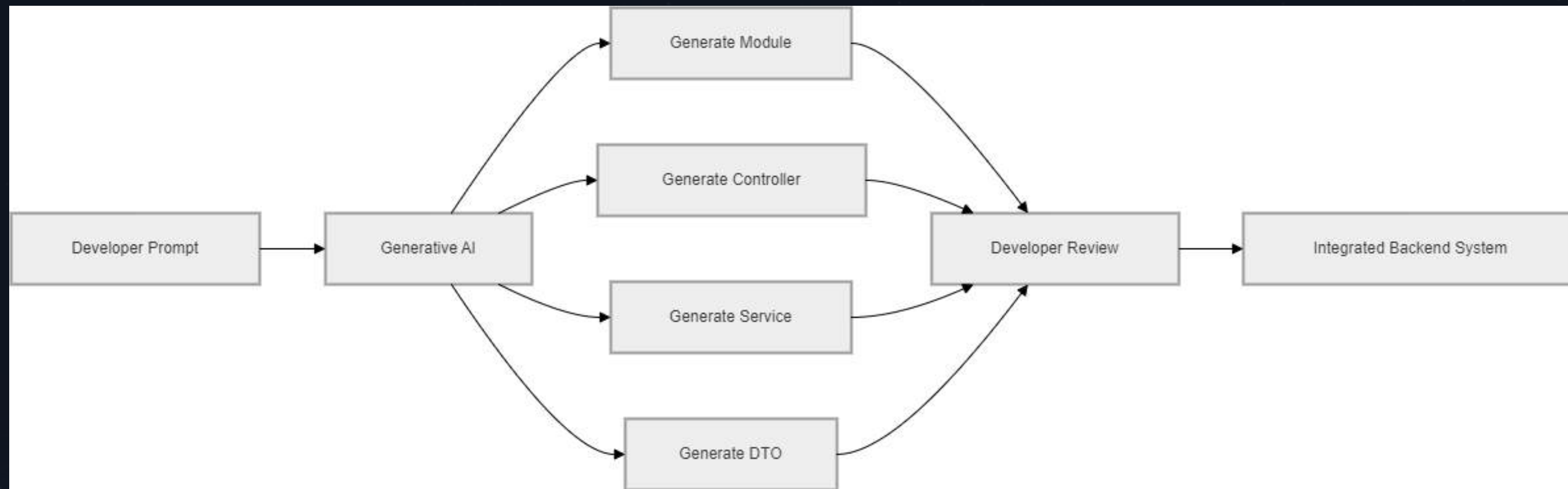
Tập trung thiết kế kiến trúc, luồng nghiệp vụ và ra quyết định chiến lược

AI

Sinh Boilerplate code, tạo cấu trúc dự án, viết code lặp lại tự động

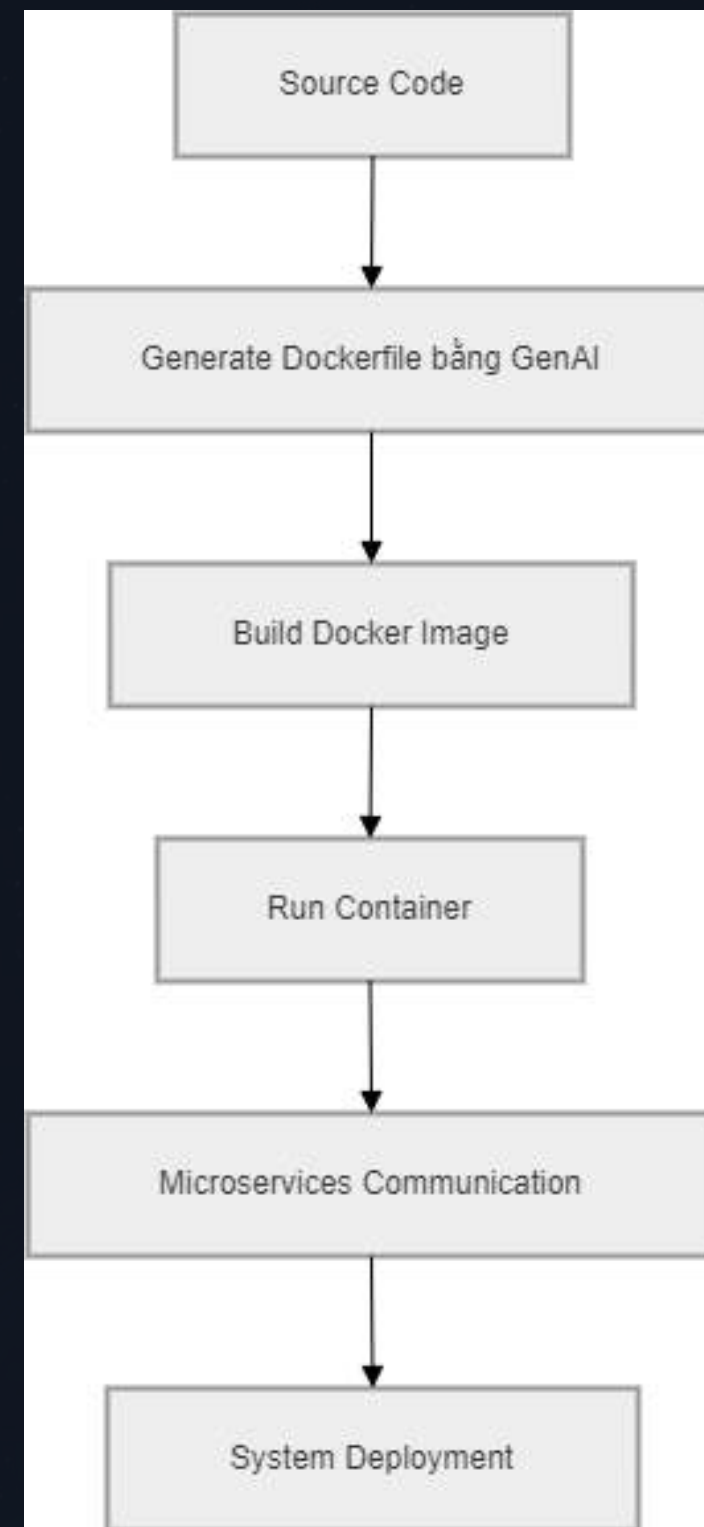
GenAI trong Vòng Đời Phát Triển (SDLC)

Quy trình tích hợp AI vào SDLC giúp tăng tốc phát triển mà vẫn đảm bảo chất lượng mã nguồn.



Quy trình này giúp giảm đáng kể thời gian viết code thủ công, cho phép developer tập trung vào chất lượng và kiến trúc hệ thống.

GenAI Hỗ Trợ Viết Mã Backend với NestJS



AI có thể tạo nhanh các thành phần cốt lõi của NestJS chỉ với một prompt mô tả nghiệp vụ:

Controller

Xử lý HTTP request, routing và response

Service

Business logic, xử lý nghiệp vụ chính

DTO

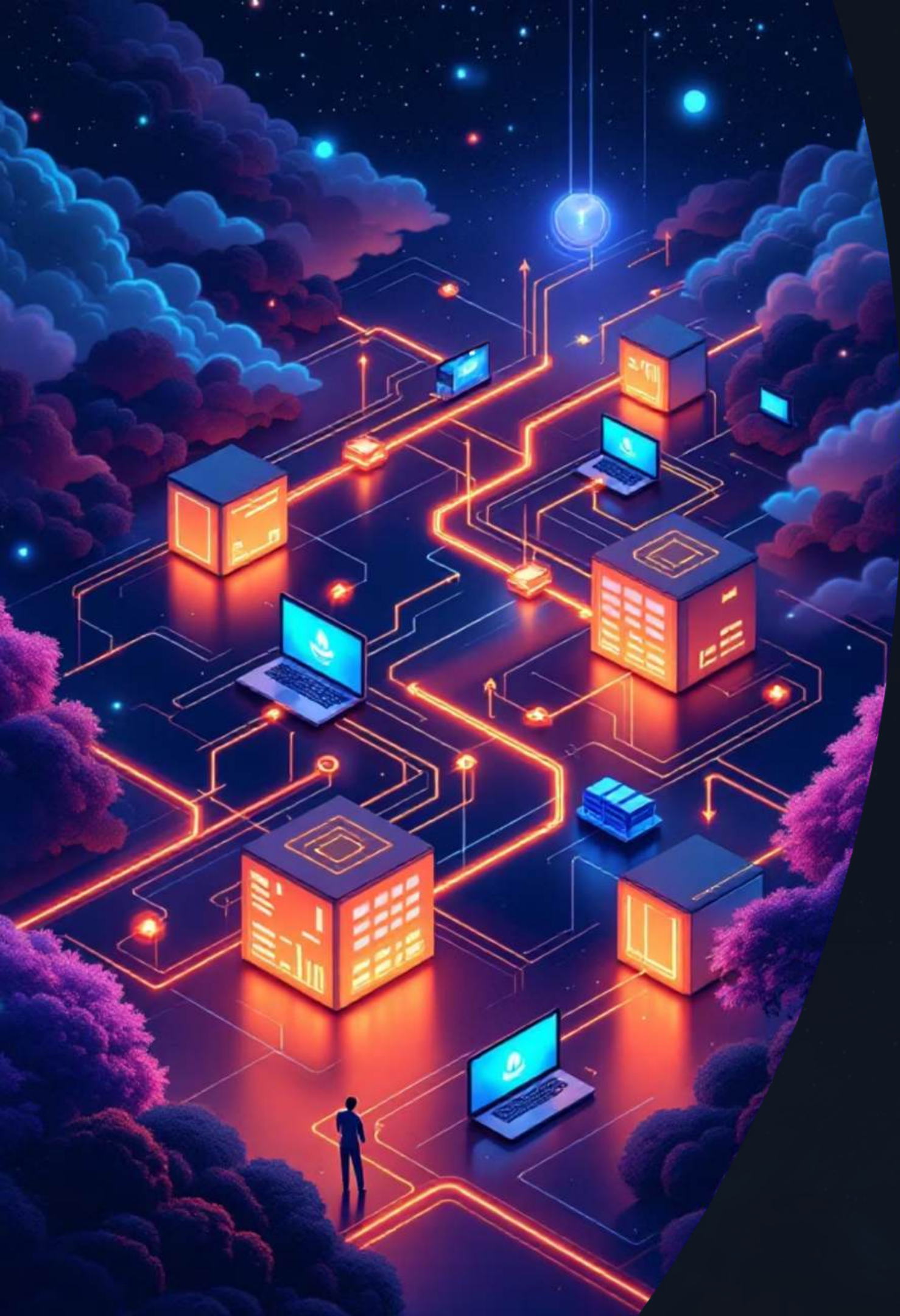
Validation và định nghĩa dữ liệu đầu vào

Prisma Schema

Thiết kế database model và migration



Chỉ với một prompt như **"Tạo Order Service với NestJS, Prisma và validation"**, AI có thể sinh ra toàn bộ cấu trúc code cơ bản trong vài giây.



Triển Khai Hệ Thống với DevOps & Docker

Sử dụng [Docker](#) để container hóa từng microservice, đảm bảo môi trường đồng nhất từ development đến production.



API Gateway

Điểm vào duy nhất, định tuyến request đến các service



Order Service

Xử lý đơn hàng, container hóa riêng biệt



Notification Service

Gửi thông báo, hoạt động độc lập qua message queue



Docker Deploy

Đồng nhất môi trường, dễ scale và rollback

Kết Quả Đạt Được

3+

Microservices

Hệ thống độc lập, có thể deploy và scale riêng biệt

~60%

Tiết kiệm thời gian

Giảm thời gian viết code lặp lại nhờ AI hỗ trợ

100%

Containerized

Toàn bộ service chạy trong Docker container



✓ Xây dựng thành công hệ thống **Microservices độc lập** với kiến trúc rõ ràng, tối ưu thời gian phát triển nhờ tích hợp AI vào quy trình coding.

Khó Khăn, Thách Thức & Hướng Phát Triển

Khó khăn & Thách thức

Logic nghiệp vụ phức tạp

AI gặp hạn chế khi xử lý các quy tắc nghiệp vụ đặc thù, đòi hỏi developer can thiệp sâu

Giao tiếp liên dịch vụ

Cross-service communication cần thiết kế cẩn thận để tránh lỗi và đảm bảo tính nhất quán

Kiểm soát lỗi chặt chẽ

Developer phải review và test kỹ code do AI sinh ra trước khi đưa vào production

Hướng Phát Triển

1 Mở rộng hệ thống

Thêm Payment Service và Inventory Service để hoàn thiện hệ sinh thái

2 Triển khai Kubernetes

Orchestration tự động, auto-scaling và high availability

3 AI Agent Monitoring

Ứng dụng AI Agent để giám sát, phát hiện lỗi và tối ưu hệ thống tự động